

【数据删除】也要学基础数据结构

Data Structure into Door

WCR²

2022 年 1 月 20 日



前言

- 由于某些不可抗力，这个专题的大部分内容都不难，所以一些简单的部分我会 gkd。
- 如果跟不上了请及时提出来。

目录

- 1 栈
 - 单调栈

2 队列

3 链表

4 桶

栈

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。

栈

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。
- 即“先进后出”。

栈

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。
- 即“先进后出”。
- 利用栈可以实现深度优先搜索 dfs。

栈

- 栈是一种特殊的线性表，特殊之处在于它只允许在表的一端进行插入和删除运算。这一端被称为栈顶，另一端称为栈底。
- 即“先进后出”。
- 利用栈可以实现深度优先搜索 dfs。
- 一般用数组模拟或者直接调用 STL 的容器 `std::stack`，但是很慢所以请谨慎使用。

括号匹配

- 判断一个括号序列是否匹配。

括号匹配

- 从左向右扫描表达式。

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；
- 如果是右括号，这个右括号应和栈顶的左括号匹配，弹出栈顶的左括号。

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；
- 如果是右括号，这个右括号应和栈顶的左括号匹配，弹出栈顶的左括号。
- 若此时栈内为空，说明当前右括号失配，返回无解信息。

括号匹配

- 从左向右扫描表达式。
- 如果是左括号，则将其压入栈中；
- 如果是右括号，这个右括号应和栈顶的左括号匹配，弹出栈顶的左括号。
- 若此时栈内为空，说明当前右括号失配，返回无解信息。
- 扫描结束时，若栈非空，说明有左括号失配，返回无解信息。

表达式求值

- 给出一个合法的仅包含数字、四则运算符和小括号的表达式，求出它的值。

表达式求值

- 开始先在表达式前后加上一对括号方便计算

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即

$$now \leftarrow now \times 10 + a_i$$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。
- 若当前字符为左括号，直接将其压入符号栈。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。
- 若当前字符为左括号，直接将其压入符号栈。
- 若当前字符为右括号，将 now 压入数字栈并清零，不断弹出两个栈内的元素直到遇到第一个左括号。

表达式求值

- 开始先在表达式前后加上一对括号方便计算
- 开两个栈，分别是符号栈和数字栈。
- 从左向右扫描表达式。
- 若当前字符为数字，则加入到当前数字末尾，即 $now \leftarrow now \times 10 + a_i$
- 若当前字符为运算符，将 now 压入数字栈并清零，将符号栈顶的所有优先级大于等于当前运算符的所有元素弹出并运算。再压入当前运算符。
- 若当前字符为左括号，直接将其压入符号栈。
- 若当前字符为右括号，将 now 压入数字栈并清零，不断弹出两个栈内的元素直到遇到第一个左括号。
- 最后数字栈内应只剩下一个元素，即为表达式的值

CSP-S2019 括号树

- 一棵树，每个节点上都有一个括号，求树上多少条直链的是合法的括号串。
- $n \leq 5 \times 10^5$

CSP-S2019 括号树

- 在树上 dfs，用栈维护当前点到根路径的括号串消除后的结果。

CSP-S2019 括号树

- 在树上 dfs，用栈维护当前点到根路径的括号串消除后的结果。
- 对于每个左括号记录一下它后面有多少个被消掉的串。加入节点时统计它产生的贡献，退出时撤销一下贡献即可。

单调栈

- 顾名思义，即栈内元素单调的栈。

单调栈

- 顾名思义，即栈内元素单调的栈。
- 一个简单的应用就是能对于一个序列中的每个数，求出它左边/右边第一个大于/小于它的数。

单调栈

- 以求左边第一个比它小的数为例。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。
- 考虑插入当前的数。如果栈顶元素大于等于当前元素，则栈顶元素显然不可能成为答案了。所以不断弹出栈顶元素，直到栈空或者栈顶元素小于当前元素。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。
- 考虑插入当前的数。如果栈顶元素大于等于当前元素，则栈顶元素显然不可能成为答案了。所以不断弹出栈顶元素，直到栈空或者栈顶元素小于当前元素。
- 此时的栈顶元素即为当前左边第一个比当前元素小的数，存储答案并在栈顶加入当前元素即可。

单调栈

- 以求左边第一个比它小的数为例。
- 从左到右扫描数组。
- 考虑插入当前的数。如果栈顶元素大于等于当前元素，则栈顶元素显然不可能成为答案了。所以不断弹出栈顶元素，直到栈空或者栈顶元素小于当前元素。
- 此时的栈顶元素即为当前左边第一个比当前元素小的数，存储答案并在栈顶加入当前元素即可。
- 时间复杂度 $O(n)$

因为太典中典了所以懒得找原题

- 求 $\sum_{i \leq j} \min a_{i \dots j}$, $n \leq 10^6$ 。

因为太典中典了所以懒得找原题

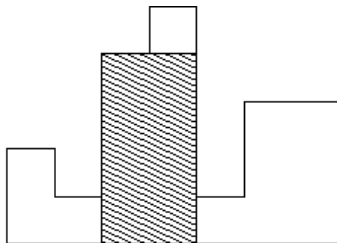
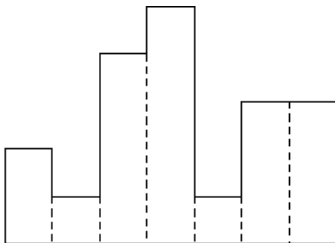
- 求 $\sum_{i \leq j} \min a_{i \dots j}$, $n \leq 10^6$ 。
- 单调栈处理出每个数 i 左右第一个比它小的位置 l_i, r_i , 那么 $\min$ 取到这个数的区间要满足 $L \in (l_i, i], R \in [i, r_i)$ 。

因为太典中典了所以懒得找原题

- 求 $\sum_{i \leq j} \min a_{i \dots j}$, $n \leq 10^6$ 。
- 单调栈处理出每个数 i 左右第一个比它小的位置 l_i, r_i , 那么 $\min$ 取到这个数的区间要满足 $L \in (l_i, i], R \in [i, r_i)$ 。
- 然后直接算贡献, 注意判一下数相同的情况即可。

poj2559 Largest Rectangle in a Histogram

- 给出一个柱状图，求其中最大的矩形的面积。
- $n \leq 10^5$



poj2559 Largest Rectangle in a Histogram

- 考虑一个最大的矩形，显然上下左右都不能再扩展。所以上边界一定会顶住某个柱子的上界。

poj2559 Largest Rectangle in a Histogram

- 考虑一个最大的矩形，显然上下左右都不能再扩展。所以上边界一定会顶住某个柱子的上界。
- 所以可以枚举矩形顶住了哪个柱子的上界，显然左右最多只能拓展到当前柱子左边右边的第一个比它低的柱子之前。计算得矩形面积更新答案即可。

poj2559 Largest Rectangle in a Histogram

- 考虑一个最大的矩形，显然上下左右都不能再扩展。所以上边界一定会顶住某个柱子的上界。
- 所以可以枚举矩形顶住了哪个柱子的上界，显然左右最多只能拓展到当前柱子左边右边的第一个比它低的柱子之前。计算得矩形面积更新答案即可。
- 至于左右第一个更低的柱子可以先用单调栈预处理得到。

作业

- NOIP2005 等价表达式
- USACO08JAN Artificial Lake G
- ZJOI2007 棋盘制作

目录

① 栈

② 队列

- 循环队列

● 双端队列

● 单调队列

③ 链表

④ 桶

队列

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。

队列

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。
- 即“先进先出”。

队列

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。
- 即“先进先出”。
- 利用队列可以实现广度优先搜索 bfs。

队列

- 队列是一种特殊的线性表，特殊之处在于它只允许在表的前端进行删除操作，而在表的后端进行插入操作。
- 即“先进先出”。
- 利用队列可以实现广度优先搜索 bfs。
- 一般用数组模拟或者直接调用 STL 的容器 `std::queue`，但是很慢所以请谨慎使用。

NOIP2016 蚯蚓

- 给出由 n 个非负整数组成的数集。每单位时间下列事件按顺序发生：
 - 将最大的数取出，设为 m ；
 - 数集中所有数字增加 q ；
 - 将 $\lfloor \frac{um}{v} \rfloor$ 和 $m - \lfloor \frac{um}{v} \rfloor$ 加入数集。
- 请求出每单位时间的 m 和 m 时刻后数集中的所有数。
- 其中 q, u, v 均为给定非负常数，且 $u < v$ 。
 $n \leq 10^5, m \leq 7 \times 10^6$ 。有某种控制输出规模的机制。

NOIP2016 蚯蚓

- 直接用堆维护得到一个 $\mathcal{O}(n \log n)$ 的 TLE 做法。考虑优化至线性。

NOIP2016 蚯蚓

- 直接用堆维护得到一个 $\mathcal{O}(n \log n)$ 的 TLE 做法。考虑优化至线性。
- 观察到当前新产生的第一、二种数一定比上一次产生的同种数小。

NOIP2016 蚯蚓

- 直接用堆维护得到一个 $\mathcal{O}(n \log n)$ 的 TLE 做法。考虑优化至线性。
- 观察到当前新产生的第一、二种数一定比上一次产生的同种数小。
- 因此可以开三个队列，分别维护初始数集、产生的第一种数、产生的第二种数排序后的结果。

NOIP2016 蚯蚓

- 直接用堆维护得到一个 $\mathcal{O}(n \log n)$ 的 TLE 做法。考虑优化至线性。
- 观察到当前新产生的第一、二种数一定比上一次产生的同种数小。
- 因此可以开三个队列，分别维护初始数集、产生的第一种数、产生的第二种数排序后的结果。
- 每次比较三个队首得到 m ，弹出并将新产生的两个数丢到两个队列的队尾。

NOIP2016 蚯蚓

- 关于性质的证明：（只讨论第一种，第二种同理）

NOIP2016 蚯蚓

- 关于性质的证明：（只讨论第一种，第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。

NOIP2016 蚯蚓

- 关于性质的证明：（只讨论第一种，第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。
- 则上次产生的数为 $m_1p + q$ ，当前产生的是 m_2p 。

NOIP2016 蚯蚓

- 关于性质的证明：（只讨论第一种，第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。
- 则上次产生的数为 $m_1p + q$ ，当前产生的是 m_2p 。
- 于是有 $m_2p \leq (m_1 + q)p = m_1p + qp$

NOIP2016 蚯蚓

- 关于性质的证明：（只讨论第一种，第二种同理）
- 设上次选的数为 m_1 ，当前选的是 m_2 。显然有 $q + m_1 \geq m_2$ 。
- 则上次产生的数为 $m_1p + q$ ，当前产生的是 m_2p 。
- 于是有 $m_2p \leq (m_1 + q)p = m_1p + qp$
- 因为 $p < 1$ ，所以有 $m_1p + q \geq m_1p + qp \geq m_2p$

循环队列

- 一个优化空间的小技巧。

循环队列

- 一个优化空间的小技巧。
- 使用数组模拟队列时我们会发现整个队列整体向数组的尾部移动，而数组前端有大量的空间空闲。

循环队列

- 一个优化空间的小技巧。
- 使用数组模拟队列时我们会发现整个队列整体向数组的尾部移动，而数组前端有大量的空间空闲。
- 所以当首尾指针越界时，令其移动到数组的头部，重新利用数组前端的空间。

循环队列

- 一个优化空间的小技巧。
- 使用数组模拟队列时我们会发现整个队列整体向数组的尾部移动，而数组前端有大量的空间空闲。
- 所以当首尾指针越界时，令其移动到数组的头部，重新利用数组前端的空间。
- 这样就相当于将数组尾部的下一个位置看作了数组的头部，故此得名“循环队列”。

双端队列

- 顾名思义，即头尾都能进行插入删除的队列。

双端队列

- 顾名思义，即头尾都能进行插入删除的队列。
- 懒的时候可以直接调用 STL 的容器 `std::deque`，但是很慢。

双端队列

- 顾名思义，即头尾都能进行插入删除的队列。
- 懒的时候可以直接调用 STL 的容器 `std::deque`，但是很慢。
- 有啥用？

我瞎编的

- 给出一张 n 个点 m 条边的边权只有 0/1 的有向图，求点 1 到点 n 的最短路。
- $n \leq 5 \times 10^6, m \leq 10^7$
- 要求复杂度严格线性。

编起来了

- 考虑双向 bfs。开始双端队列内只有一个点 1。

编起来了

- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。

编起来了

- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。
- 若当前边边权为 0，且连向的点 v 的最短路被更新，则将 v 插到队首。

编起来了

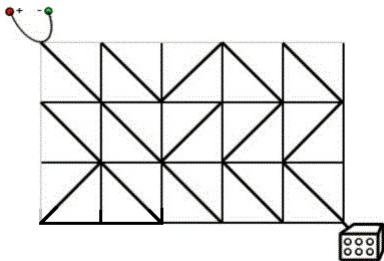
- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。
- 若当前边边权为 0，且连向的点 v 的最短路被更新，则将 v 插到队首。
- 若当前边边权为 1，且连向的点 v 的最短路被更新，则将 v 插到队尾。

编起来了

- 考虑双向 bfs。开始双端队列内只有一个点 1。
- 每次从队首取出一个元素，以它为起点进行松弛。
- 若当前边边权为 0，且连向的点 v 的最短路被更新，则将 v 插到队首。
- 若当前边边权为 1，且连向的点 v 的最短路被更新，则将 v 插到队尾。
- 正确性显然。

BalticOI 2011 Day1 Switch the Lamp On

- 有 $n \times m$ 个正方形电路元件，排列成 n 行 m 列。
- 每个电路元件连通了它的一条对角线。
- 每次操作你可以使一个电路元件旋转 90° 。
- 位于 $(1,1)$ 的元件的左上角连接了电源，位于 (n,m) 的元件的右下角连接了灯泡。
- 求最少操作多少次使得电源与灯连通， $n, m \leq 10^3$ 。



BalticOI 2011 Day1 Switch the Lamp On

- 建 $(n + 1) \times (m + 1)$ 个点，表示网格图的每个格点。

BalticOI 2011 Day1 Switch the Lamp On

- 建 $(n + 1) \times (m + 1)$ 个点，表示网格图的每个格点。
- 对于每个元件的每条对角线，若已经连通则连边权为 0 的边，否则连边权为 1 的边，表示需要操作一次。

BalticOI 2011 Day1 Switch the Lamp On

- 建 $(n + 1) \times (m + 1)$ 个点，表示网格图的每个格点。
- 对于每个元件的每条对角线，若已经连通则连边权为 0 的边，否则连边权为 1 的边，表示需要操作一次。
- 然后直接跑双向 bfs 求出 $(1, 1)$ 到 $(n + 1, m + 1)$ 的最短路即可

单调队列

- 顾名思义，即队内元素单调的队列。

单调队列

- 顾名思义，即队内元素单调的队列。
- 要注意的是它本质上其实是双端队列。

单调队列

- 顾名思义，即队内元素单调的队列。
- 要注意的是它本质上其实是双端队列。
- 可以解决一类 rmq 问题。

单调队列

- 顾名思义，即队内元素单调的队列。
- 要注意的是它本质上其实是双端队列。
- 可以解决一类 `rmq` 问题。
- 具体是啥？先来道例题。

poj2823 Sliding Window

- 给出一个长度为 n 的序列 a 和整数 k 。
- 定义 $b_i = \min_{j=i}^{i+k-1} a_j$ 。
- 定义 $c_i = \max_{j=i}^{i+k-1} a_j$ 。
- 求 $b_{1..n-k+1}, c_{1..n-k+1}$ 。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不优，不断弹出队尾元素直到队空或队尾元素小于当前元素。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不优，不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不优，不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不优，不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。
- 队首元素显然就是当前区间的最小值。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不优，不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。
- 队首元素显然就是当前区间的最小值。
- 但此时队首元素可能已经不在当前区间中了，因此需要判断队首元素若不在询问区间内则不断弹出队首直到合法。

poj2823 Sliding Window

- 考虑维护一个队内元素递增的队列。队内元素即为当前区间最优元素的候选队列。
- 以求 b 序列为例。从左到右扫描数组。
- 如果队尾元素大于等于当前元素，那么队尾元素显然不优，不断弹出队尾元素直到队空或队尾元素小于当前元素。
- 此时向队尾插入当前元素。
- 可以发现队列内的元素是单调递增的。
- 队首元素显然就是当前区间的最小值。
- 但此时队首元素可能已经不在当前区间中了，因此需要判断队首元素若不在询问区间内则不断弹出队首直到合法。
- 可以发现使用单调队列的条件是询问区间的左右端点都单调不降。

CF1077F Pictures with Kittens

- 给出一个长度为 n 的序列 a ，你需要选择 m 个元素，使得每连续的 k 个元素中至少有一个数被选中。
- 求选中的所有数的和的最大值。
- $1 \leq k, m \leq n \leq 5000, 1 \leq a_i \leq 10^9$

CF1077F Pictures with Kittens

- 单调队列优化 dp 板子题。

CF1077F Pictures with Kittens

- 单调队列优化 dp 板子题。
- 设 $f_{i,j}$ 表示前 i 个数里选了 j 个数，强制第 i 个数选的最大和。

CF1077F Pictures with Kittens

- 单调队列优化 dp 板子题。
- 设 $f_{i,j}$ 表示前 i 个数里选了 j 个数，强制第 i 个数选的最大和。
- 显然有 $f_{i,j} = \max_{t=i-k}^{i-1} f_{t,j-1} + a_i$ 。

CF1077F Pictures with Kittens

- 单调队列优化 dp 板子题。
- 设 $f_{i,j}$ 表示前 i 个数里选了 j 个数，强制第 i 个数选的最大和。
- 显然有 $f_{i,j} = \max_{t=i-k}^{i-1} f_{t,j-1} + a_i$ 。
- 先枚举 j 再枚举 i ，然后 $\max$ 里的东西就可以单调队列优化转移了。

POI2014 PTA-Little Bird

- 有 n 棵树，第 i 棵树的高度是 d_i 。
- 邪王真眼开始在第一棵树，她要去第 n 棵树。
- 如果邪王真眼在第 i 棵树，那么她可以跳到第 $i+1, i+2, \dots, i+k$ 棵树。
- 如果她跳到一棵不矮于当前树的树，那么她的劳累值会增加 1；否则不会。
- 求邪王真眼到达第 n 棵树的最小劳累值。
- q 组询问，每组给出不同的 k
- $n \leq 10^6, q \leq 25$ 。

POI2014 PTA-Little Bird

- 设 f_i 表示走到第 i 个点最小的疲劳值。

POI2014 PTA-Little Bird

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。

POI2014 PTA-Little Bird

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。
- 原因是两个疲劳值不同的显然是疲劳值较小的更优，因为疲劳值最多只会加 1。

POI2014 PTA-Little Bird

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。
- 原因是两个疲劳值不同的显然是疲劳值较小的更优，因为疲劳值最多只会加 1。
- 而在疲劳值相同时显然是树高更高的更优。

POI2014 PTA-Little Bird

- 设 f_i 表示走到第 i 个点最小的疲劳值。
- 考虑怎样的转移点是最优的。可以发现是优先疲劳值最小，在疲劳值相同时保证高度最大。
- 原因是两个疲劳值不同的显然是疲劳值较小的更优，因为疲劳值最多只会加 1。
- 而在疲劳值相同时显然是树高更高的更优。
- 按照这个策略用单调队列维护即可。

LOJ6515 「雅礼集训 2018 Day10」贪玩蓝月

- 维护一个二元组 (w, v) 的双端队列，每次操作支持插入、删除或者查询。
- 查询给出 l, r ，要求在当前的 deque 中选出若干个二元组使得 $(\sum w) \bmod p \in [l, r]$ ，且 $\sum v$ 最大。
- $Q \leq 5 \times 10^4, p \leq 500$

LOJ6515 「雅礼集训 2018 Day10」贪玩蓝月

- 可以发现只有一端有操作可以用栈简单维护。

LOJ6515 「雅礼集训 2018 Day10」贪玩蓝月

- 可以发现只有一端有操作可以用栈简单维护。
- 有个不带删尺取的套路是定一个 mid ，然后维护两边的答案。

LOJ6515 「雅礼集训 2018 Day10」贪玩蓝月

- 可以发现只有一端有操作可以用栈简单维护。
- 有个不带删尺取的套路是定一个 mid ，然后维护两边的答案。
- 考虑维护 mid 两边的栈，两个栈合并的答案是可以单调队列做到 $\mathcal{O}(p)$ 查询的。

LOJ6515 「雅礼集训 2018 Day10」贪玩蓝月

- 可以发现只有一端有操作可以用栈简单维护。
- 有个不带删尺取的套路是定一个 mid ，然后维护两边的答案。
- 考虑维护 mid 两边的栈，两个栈合并的答案是可以单调队列做到 $\mathcal{O}(p)$ 查询的。
- 现在问题是弹出 mid 时如何处理，此时直接将 mid 定为当前 deque 的中点然后暴力重构。

LOJ6515 「雅礼集训 2018 Day10」贪玩蓝月

- 可以发现只有一端有操作可以用栈简单维护。
- 有个不带删尺取的套路是定一个 mid ，然后维护两边的答案。
- 考虑维护 mid 两边的栈，两个栈合并的答案是可以单调队列做到 $\mathcal{O}(p)$ 查询的。
- 现在问题是弹出 mid 时如何处理，此时直接将 mid 定为当前 deque 的中点然后暴力重构。
- 分析一波复杂度可以发现修改均摊下来是 $\mathcal{O}(p \log Q)$ 的。

作业

- HAOI2007 理想的正方形
- POI2011 TEM-Temperature
- CSP-S2019 划分

目录

① 栈

② 队列

③ 链表

- 二维链表
- 块状链表

④ 桶

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。
- 链表由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储相邻结点地址的指针域。

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。
- 链表由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储相邻结点地址的指针域。
- 分为单向链表和双向链表。

链表

- 链表是一种物理存储单元上非连续、非顺序的存储结构，数据元素的逻辑顺序是通过链表中的指针链接次序实现的。
- 链表由一系列结点组成，每个结点包括两个部分：一个是存储数据元素的数据域，另一个是存储相邻结点地址的指针域。
- 分为单向链表和双向链表。
- 一个常用的用法是链式前向星存图。

链表

- 链表也可以求每个数它左右第一个大于/小于它的数。

链表

- 链表也可以求每个数它左右第一个大于/小于它的数。
- 先开一个原序列的链表，从大到小删数，每次删数时左右的数即为第一个比它小的数。

链表

- 链表也可以求每个数它左右第一个大于/小于它的数。
- 先开一个原序列的链表，从大到小删数，每次删数时左右的数即为第一个比它小的数。
- 因为比它大的数已经被删光了，所以正确性显然。

链表

- 链表也可以求每个数它左右第一个大于/小于它的数。
- 先开一个原序列的链表，从大到小删数，每次删数时左右的数即为第一个比它小的数。
- 因为比它大的数已经被删光了，所以正确性显然。
- 因为单调栈很好写而且有重复的数时会会有点麻烦所以一般没人写。

CF706E Working routine

- 给出一个 $n \times m$ 的矩阵和 Q 次操作，每次操作交换两个不重合且不相邻的两个完全相同的矩形中的所有元素，求最后的结果。
- $n, m \leq 1000, Q \leq 10000$ 。

CF706E Working routine

- 二维链表即可，时间复杂度 $O((n + m)Q)$

块状链表

- 众所周知

块状链表

- 众所周知
- 链表插入删除是 $\mathcal{O}(1)$ 的，但随机访问是 $\mathcal{O}(n)$ 的。

块状链表

- 众所周知
- 链表插入删除是 $\mathcal{O}(1)$ 的，但随机访问是 $\mathcal{O}(n)$ 的。
- 数组随机访问是 $\mathcal{O}(1)$ 的，但插入删除是 $\mathcal{O}(n)$ 的。

块状链表

- 众所周知
- 链表插入删除是 $\mathcal{O}(1)$ 的，但随机访问是 $\mathcal{O}(n)$ 的。
- 数组随机访问是 $\mathcal{O}(1)$ 的，但插入删除是 $\mathcal{O}(n)$ 的。
- 而块状链表是把这两个数据结构 上上下下左右左右 BABA 一下，两个操作都是 $\mathcal{O}(\sqrt{n})$ 。

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当块长度超过 $2B$ 时分裂当前结点。

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当块长度超过 $2B$ 时分裂当前结点。
- 最多分裂 $\frac{n}{B}$ 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当块长度超过 $2B$ 时分裂当前结点。
- 最多分裂 $\frac{n}{B}$ 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。
- 显然 B 取 $\sqrt{n}$ 时最优。

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当块长度超过 $2B$ 时分裂当前结点。
- 最多分裂 $\frac{n}{B}$ 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。
- 显然 B 取 $\sqrt{n}$ 时最优。
- 模板题：poj2887 Big String

块状链表

- 设置一个阈值 B ，维护一个每个元素都是一个长度 $\mathcal{O}(B)$ 的数组的链表。
- 插入删除时遍历链表找到位置并暴力修改，复杂度 $\mathcal{O}(\max(B, \frac{n}{B}))$ 。
- 但是修改完我们需要保证每个结点内数组的长度接近 B ，所以可以当块长度超过 $2B$ 时分裂当前结点。
- 最多分裂 $\frac{n}{B}$ 次，总时间复杂度 $\mathcal{O}(\frac{n}{B} \times B)$ 。
- 显然 B 取 $\sqrt{n}$ 时最优。
- 模板题：poj2887 Big String
- 有什么用？妈妈再也不用担心我不会写平衡树了!! 1。

目录

① 栈

② 队列

③ 链表

④ 桶

桶

- 根据数值直接进行访问的数据结构。

桶

- 根据数值直接进行访问的数据结构。
- 我不会的应用：桶排序、基数排序。

BJWC2010 取数游戏

- 给出一个长度为 n 的数列，数列中的数字不超过 m 。要求在原序列中取出一个尽量长的子序列，使子序列中相邻两个数的最大公约数全部大于 L 。
- $n \leq 50000, m \leq 1000000$ 。

BJWC2010 取数游戏

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。

BJWC2010 取数游戏

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。
- 维护一个桶，第 i 个位置表示包含 i 这个因子的数最大的 f 值是多少。

BJWC2010 取数游戏

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。
- 维护一个桶，第 i 个位置表示包含 i 这个因子的数最大的 f 值是多少。
- 转移时枚举因子，从桶里找最大值即可。

BJWC2010 取数游戏

- 设 f_i 表示强制选第 i 个数，前 i 个数中最多能选出多少数。
- 维护一个桶，第 i 个位置表示包含 i 这个因子的数最大的 f 值是多少。
- 转移时枚举因子，从桶里找最大值即可。
- 转移完再将当前数加入桶中，同样枚举因子即可。

……那、那個
……要笑的話就笑吧。」

GL&HF
Thanks for Listening